

磁盘取数据要用一段相对较长的时间(数量级为几十毫秒),所以内核执行从进程 A 到进程 B 的上下文切换,而不是在这个间歇时间内等待,什么都不做。注意在切换之前,内核正代表进程 A 在用户模式下执行指令(即没有单独的内核进程)。在切换的第一部分中,内核代表进程 A 在内核模式下执行指令。然后在某一时刻,它开始代表进程 B(仍然是内核模式下)执行指令。在切换之后,内核代表进程 B 在用户模式下执行指令。

随后,进程 B 在用户模式下运行一会儿,直到磁盘发出一个中断信号,表示数据已经从磁盘传送到了内存。内核判定进程 B 已经运行了足够长的时间,就执行一个从进程 B 到进程 A 的上下文切换,将控制返回给进程 A 中紧随在系统调用 read 之后的那条指令。进程 A 继续运行,直到下一次异常发生,依此类推。

8.3 系统调用错误处理

当 Unix 系统级函数遇到错误时,它们通常会返回 -1,并设置全局整数变量 `errno` 来表示什么出错了。程序员应该总是检查错误,但是不幸的是,许多人都忽略了错误检查,因为它使代码变得臃肿,而且难以读懂。比如,下面是我们调用 Unix `fork` 函数时会如何检查错误:

```
1      if ((pid = fork()) < 0) {
2          fprintf(stderr, "fork error: %s\n", strerror(errno));
3          exit(0);
4      }
```

`strerror` 函数返回一个文本串,描述了和某个 `errno` 值相关联的错误。通过定义下面的错误报告函数,我们能够在某种程度上简化这个代码:

```
1  void unix_error(char *msg) /* Unix-style error */
2  {
3      fprintf(stderr, "%s: %s\n", msg, strerror(errno));
4      exit(0);
5  }
```

给定这个函数,我们对 `fork` 的调用从 4 行缩减到 2 行:

```
1      if ((pid = fork()) < 0)
2          unix_error("fork error");
```

通过使用错误处理包装函数,我们可以更进一步地简化代码,Stevens 在[110]中首先提出了这种方法。对于一个给定的基本函数 `foo`,我们定义一个具有相同参数的包装函数 `Foo`,但是第一个字母大写了。包装函数调用基本函数,检查错误,如果有任何问题就终止。比如,下面是 `fork` 函数的错误处理包装函数:

```
1  pid_t Fork(void) => wrapper function, 其本质是将系统级函数的调用,
2  {                                     错误检查与错误处理封装于一个 function 内部的操作
3      pid_t pid;                       遵循“必须检查返回值”的规律。
4
5      if ((pid = fork()) < 0)
6          unix_error("Fork error");
7      return pid;
8  }
```

给定这个包装函数,我们对 `fork` 的调用就缩减为 1 行: